

Lp-3 DAA_Practical

Practical-1

Non-recursive

```
#include <iostream>
```

```
using namespace std;
```

```
int fibonacciIterative(int n, int &steps) {
```

```
    steps = 0;
```

```
    if (n == 0) {
```

```
        steps++;
```

```
        return 0;
```

```
    }
```

```
    if (n == 1) {
```

```
        steps++;
```

```
        return 1;
```

```
    }
```

```
    int a = 0, b = 1, c;
```

```
    for (int i = 2; i <= n; i++) {
```

```
        c = a + b;
```

```
        a = b;
```

```
        b = c;
```

```
        steps++; // Count each iteration
```

```
    }
```

```
    return b;
```

```
}
```

```
int main() {
```

```
    int n, steps;
```

```
    cout << "Enter n: ";
```

```
    cin >> n;
```

```
    int result = fibonacciIterative(n, steps);
```

```

    cout << "Fibonacci(" << n << ") = " << result << endl;

    cout << "Step count (iterative): " << steps << endl;

    return 0;
}

Recursive

#include <iostream>

using namespace std;

int fibonacciRecursive(int n, int &steps) {
    steps++; // Count each function call

    if (n == 0)
        return 0;

    if (n == 1)
        return 1;

    return fibonacciRecursive(n - 1, steps) + fibonacciRecursive(n - 2, steps);
}

int main() {
    int n, steps = 0;

    cout << "Enter n: ";

    cin >> n;

    int result = fibonacciRecursive(n, steps);

    cout << "Fibonacci(" << n << ") = " << result << endl;

    cout << "Step count (recursive): " << steps << endl;

    return 0;
}

```

Practical-2

```

#include<bits/stdc++.h>

using namespace std;

struct Node{

```

```

char ch;

int freq;

Node*left;

Node*right;


Node(char c,int f){

    ch=c;

    freq=f;

    left=right=nullptr;

}

};


struct Compare{

    bool operator()(Node*left,Node*right){

        return left->freq > right->freq;

    }

};


void printtree(Node* root,string str){

    if(!root){

        return;

    }

    if(!root->left && !root->right){

        cout<<root->ch<<":"<<str<<"\n";

        return;

    }

    printtree(root->left,str+"0");

    printtree(root->right,str+"1");

}


void hufmann(const string& text){

    unordered_map<char,int> freq;

    for(char s:text){

```

```

        freq[s]++;
    }

    priority_queue<Node*,vector<Node*>,Compare> minheap;

    for(auto it:freq){
        minheap.push(new Node(it.first,it.second));
    }

    while(minheap.size()>1){
        Node*left=minheap.top();
        minheap.pop();
        Node*right=minheap.top();
        minheap.pop();

        Node*newNode=new Node('$',left->freq+right->freq);
        newNode->left=left;
        newNode->right=right;

        minheap.push(newNode);
    }
    Node*root=minheap.top();
    cout<<"Huffmann Coding\n";
    printtree(root,"");

}

int main(){
    string text;
    cout<<"Enter the string:";
    cin>>text;
    hufmann(text);
}

```

```
}
```

Practical-3

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
struct Item{
```

```
    int weight;
```

```
    int value;
```

```
    Item(int w,int v){
```

```
        weight=w;
```

```
        value=v;
```

```
    }
```

```
};
```

```
bool cmp(Item a,Item b){
```

```
    double r1=(double)a.value/a.weight;
```

```
    double r2=(double)b.value/b.weight;
```

```
    return r1>r2;
```

```
}
```

```
int knapsack(int W,vector<Item> &arr){
```

```
    sort(arr.begin(),arr.end(),cmp);
```

```
    int curItem=0;
```

```
    double finalvalue=0.0;
```

```
    for(auto item :arr){
```

```
        if(curItem+item.weight <=W){
```

```
            curItem+=item.weight;
```

```
            finalvalue+=item.value;
```

```
        }
```

```
    else{
```

```
        int remain=W-curItem;
```

```

        finalvalue+=item.value*((double)remain/item.weight);

        break;
    }
}

return finalvalue;
}

```

```

int main(){
    int n;

    cout<<"Enter no of items";

    cin>>n;

    vector<Item> arr;
    for(int i=0;i<n;i++){
        int w;

        int v;

        cout<<"Weight";

        cin>>w;

        cout<<"Value";

        cin>>v;

        arr.push_back(Item(w,v));
    }

    int W;

    cout<<"Enter the W weight";

    cin>>W;

    cout<<knapsack(W,arr);
}

```

Practical-4

```

#include<bits/stdc++.h>

using namespace std;

```

```

int knapsack(int W,vector<int> val,vector<int> wt,int n){
    vector<vector<int>> dp(n+1,vector<int>(W+1,0));
}

```

```

for(int i=1;i<=n;i++){
    for(int j=1;j<=W;j++){
        if(wt[i-1]<=j){
            dp[i][j]=max(dp[i-1][j],val[i-1]+dp[i-1][j-wt[i-1]]);
        }
        else{
            dp[i][j]=dp[i-1][j];
        }
    }
}
return dp[n][W];
}

```

```

int main(){
    int n;

    cout<<"Enter the items";

    cin>>n;

    vector<int> val(n),wt(n);

    cout<<"Enter the values";
    for(int i=0;i<n;i++){
        cin>>val[i];
    }

    cout<<"Enter the weights";
    for(int i=0;i<n;i++){
        cin>>wt[i];
    }

    int W;

    cout<<"Enter the W";

    cin>>W;

```

```
cout<<knapsack(W,val,wt,n);
```

```
}
```

Practical-5

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int piv_sort(vector<int> &arr,int l,int h){
```

```
    int pivot=arr[l];
```

```
    int i=l;
```

```
    int j=h;
```

```
    while(i<j){
```

```
        while(arr[i]<=pivot && i<=h-1){
```

```
            i++;
```

```
        }
```

```
        while(arr[j]>pivot && j>=l+1){
```

```
            j--;
```

```
        }
```

```
        if(i<j) swap(arr[i],arr[j]);
```

```
    }
```

```
    swap(arr[l],arr[j]);
```

```
    return j;
```

```
}
```

```
void qs(vector<int> &arr,int l,int h){
```

```
    if(l<h){
```

```
        int piv=piv_sort(arr,l,h);
```

```
        qs(arr,l,piv-1);
```

```
        qs(arr,piv+1,h);
```

```
    }
```

```
}
```



```
int main(){  
    vector<int> arr={6,4,2,8,2,0,1};  
    int n=arr.size();  
    qs(arr,0,n-1);  
    for(int it:arr){  
        cout<<it<<" ";  
    }  
}
```